

OPTIMISATION DE MODELES NEURONAUX A OSCILLATEURS COUPLÉS - LIVRABLE 3 : ÉTAT DE L'ART SOUS FORME DE BENCHMARK

DA SILVA Tristan - BATTAGLIA Charles-Antoine
BOTTÉ-MAGALHAES Jules - GAUTHIER Keanu - FRAYSSE Ianis

JUNIA - ISEN - AP4
<https://github.com/CharlesBta/Modele-Neuronaux-RD.git>

Lexique

Backend		Moteur d'exécution d'un simulateur ; Brian2CUDA ajoute par exemple un backend GPU au simulateur Brian.
Génération de code		Production automatique du code GPU à partir d'une description de modèle écrite en langage de haut niveau (Python ou Julia).
JAX		Bibliothèque Python de calcul différentiable avec vectorisation automatique (vmap).
Jonction cavante	communi-	<i>Gap junction</i> : couplage électrique direct et bidirectionnel entre deux cellules voisines.
Julia		Langage de programmation scientifique haut niveau, proche du C++ en performance.
MPI		<i>Message Passing Interface</i> : norme d'échange de messages entre machines, utilisée pour répartir une simulation sur plusieurs nœuds de calcul.
NEST		Simulateur de référence pour les grandes populations de neurones simplifiés.
NVLink		Liaison matérielle à très haut débit entre GPU NVIDIA .
PyTorch		Bibliothèque Python d'apprentissage automatique, aussi utilisée pour du calcul GPU généraliste.
vmap		Opération de vectorisation automatique de JAX et PyTorch : une même fonction s'applique à un grand nombre d'éléments en parallèle.

Table des matières

Lexique	1
1 Fiches d'analyse des articles	3
1.1 Scalable Parallel Programming with CUDA (2008)	3
1.2 GeNN : a code generation framework for accelerated brain simulations (2016)	4
1.3 Brian2CUDA : Flexible and Efficient Simulation of Spiking Neural Network Models on GPUs (2022)	4
1.4 Automated Translation and Accelerated Solving of Differential Equations on Multiple GPU Platforms (2024)	5
1.5 SimHH : A Versatile, Multi-GPU Simulator for Extended Hodgkin-Huxley Networks (2025)	5
2 Benchmark comparatif	6
Bibliographie	7

Fiches d'analyse des articles

1.1 Scalable Parallel Programming with CUDA (2008)

Cet article fondateur, écrit par des ingénieurs **NVIDIA** et un universitaire [1], explique le modèle de programmation **CUDA** au moment où les **CPU** deviennent multicœurs et les **GPU** massivement parallèles.

Pourquoi ?	Les processeurs grand public gagnent désormais en nombre de cœurs plutôt qu'en fréquence : le matériel devient parallèle, et les logiciels classiques ne le sont pas. Le problème posé : écrire des programmes qui passent à l'échelle quel que soit le nombre de cœurs.
Comment ?	Le modèle CUDA repose sur trois abstractions : une hiérarchie de groupes de fils d'exécution, des mémoires partagées et des barrières de synchronisation. Le programmeur répartit le travail en blocs de fils sans connaître le nombre de cœurs : le matériel s'occupe de l'ordonnancement.
Avec quoi ?	Cartes GPU NVIDIA et kit de développement CUDA , illustrés par des exemples d'applications scientifiques parallèles.
Quels résultats ?	Un même programme CUDA s'exécute sur des GPU de quelques dizaines ou de plusieurs centaines de cœurs sans être réécrit : le parallélisme suit le matériel.
Quelles limites ?	Article de présentation de 2008 : matériel et outils datés, pas de mesure sur des EDO biologiques, et le programmeur travaille toujours à un niveau bas (gestion fine de la mémoire et des fils).
Pour notre sujet ?	C'est le socle conceptuel du projet : paralléliser Hodgkin-Huxley voudra dire répartir les neurones sur des fils d'exécution et organiser la mémoire partagée pour le couplage synaptique.

1.2 GeNN : a code generation framework for accelerated brain simulations (2016)

GeNN (*GPU-enhanced Neuronal Networks*) est une bibliothèque open source développée à l'université de Sussex pour mettre les **GPU** à portée des neuroscientifiques [2].

Pourquoi ?	Simuler des réseaux de neurones réalistes est lent, et les accélérateurs GPU restent inaccessibles à ceux qui ne savent pas programmer à bas niveau.
Comment ?	L'utilisateur décrit ses neurones et ses synapses dans une interface de haut niveau ; GeNN génère automatiquement le code CUDA correspondant, sans exiger de connaissances GPU approfondies.
Avec quoi ?	GPU NVIDIA ; bancs d'essai sur plusieurs modèles de réseaux, dont un réseau d'un million de neurones de Hodgkin-Huxley à conductance.
Quels résultats ?	Jusqu'à 200 fois plus rapide qu'un cœur de CPU pour le réseau d'un million de neurones Hodgkin-Huxley ; mais le gain change selon le modèle simulé.
Quelles limites ?	Gain très dépendant du modèle ; comparaison faite contre un seul cœur de CPU , donc une référence faible ; NVIDIA uniquement.
Pour notre sujet ?	Preuve directe qu'un réseau de Hodgkin-Huxley à conductance passe à l'échelle sur GPU , et première forme de l'idée de génération de code que notre projet pourra réutiliser ou comparer à des noyaux écrits à la main.

1.3 Brian2CUDA : Flexible and Efficient Simulation of Spiking Neural Network Models on GPUs (2022)

Brian2CUDA ajoute un backend **GPU** au simulateur Brian2, très utilisé en neurosciences computationnelles [3].

Pourquoi ?	Les chercheurs décrivent leurs modèles en Python avec le simulateur Brian2, mais celui-ci n'avait pas de sortie GPU complète : écrire le code bas niveau soi-même demandait un savoir-faire rare.
Comment ?	Le simulateur génère lui-même le code CUDA à partir de la définition du modèle en Python : un volet pour l'intégration numérique des états des neurones, un autre pour la propagation des événements synaptiques.
Avec quoi ?	Code CUDA généré ; comparaisons avec le backend CPU de Brian sur plusieurs types de modèles, y compris neurones et synapses arbitraires et plasticité.
Quels résultats ?	Jusqu'à trois ordres de grandeur d'accélération face au backend CPU de Brian ; seule solution GPU qui couvre toutes les fonctionnalités de Brian2.
Quelles limites ?	Lié au cadre Brian2 : on doit écrire ses modèles dans ce simulateur ; gain variable selon les modèles ; un seul GPU .
Pour notre sujet ?	Démontre qu'on peut accélérer sans écrire de CUDA à la main : le modèle décrit en Python est traduit. Notre socle Python /SciPy pourrait suivre la même voie : compiler les équations plutôt que réécrire le solveur.

1.4 Automated Translation and Accelerated Solving of Differential Equations on Multiple GPU Platforms (2024)

Cet article, publié dans Computer Methods in Applied Mechanics and Engineering [4], cherche à résoudre beaucoup d'**EDO** sur **GPU** sans écrire de code compliqué, et quel que soit le constructeur de la carte.

Pourquoi ?	Résoudre beaucoup d' EDO sur GPU oblige normalement à écrire du code CUDA à la main, et ce code ne fonctionne que sur les cartes NVIDIA .
Comment ?	Le solveur d' EDO du langage Julia traduit tout seul le code pour la carte : on écrit ses équations comme d'habitude, et elles s'exécutent sur le GPU sans rien changer au modèle.
Avec quoi ?	Des GPU de quatre constructeurs (NVIDIA , AMD, Intel, Apple), et des comparaisons avec du CUDA-C++ écrit à la main puis avec JAX et PyTorch .
Quels résultats ?	Aussi rapide que du CUDA-C++ optimisé à la main, et 20 à 100 fois plus rapide que la solution de JAX et PyTorch . Le même code tourne sur toutes les cartes testées.
Quelles limites ?	Les EDO de l'article sont indépendantes les unes des autres : pas de synapses ni de connectome, donc pas de couplage irrégulier comme dans notre cas. Et il faut travailler en Julia .
Pour notre sujet ?	Notre réseau est justement un gros système d' EDO . L'article montre qu'on peut éviter d'écrire le CUDA nous-mêmes et ne pas dépendre d'un seul constructeur. Reste à vérifier si ça marche aussi bien quand les équations sont couplées entre elles.

1.5 SimHH : A Versatile, Multi-GPU Simulator for Extended Hodgkin-Huxley Networks (2025)

SimHH est le travail le plus récent et le plus proche de notre cas : un simulateur de réseaux de **Hodgkin-Huxley** étendus, distribué sur des clusters de calcul [5].

Pourquoi ?	Les réseaux de Hodgkin-Huxley étendus (multi-compartiments, jonctions communicantes) dépassent ce qu'une seule machine peut simuler en un temps raisonnable.
Comment ?	Répartition de la simulation sur plusieurs nœuds de calcul avec MPI (OpenMPI), avec deux configurations : partager tous les potentiels entre nœuds, ou ne partager que les potentiels utiles ; CUDA et NVLink pour le calcul, les noyaux tournant en parallèle des communications.
Avec quoi ?	Clusters multi- GPU allant jusqu'à 256 nœuds ; modèles étendus dont le noyau inféro-olivaire ; comparaisons avec des CPU , un FPGA et le simulateur NEST .
Quels résultats ?	Réseaux de plus de 10 millions de cellules ; environ 150× plus rapide qu'un CPU mono-fil, 10× qu'un CPU à 128 fils, 10× qu'un FPGA , et 7× que NEST sur 32 nœuds pour un réseau de 94 720 neurones à jonctions communicantes.
Quelles limites ?	Exige un cluster de calcul (plusieurs machines reliées) et MPI : une infrastructure bien au-delà d'un serveur universitaire ; la configuration de répartition est délicate à régler.
Pour notre sujet ?	État de l'art le plus proche : Hodgkin-Huxley , comparaisons honnêtes (CPU multi-fils, FPGA , simulateurs) et preuve qu'un seul GPU finit par saturer. Nos mesures se placeront juste avant cette étape multi- GPU .

Benchmark comparatif

Le tableau ci-dessous compare les cinq approches sur les critères de benchmarking. Les gains de performance sont indiqués par rapport à la référence mesurée dans chaque article.

Outil	Type	Technicité	Maturité (TRL)	Coût	Facilité	Performance	Moy.
CUDA [1]	Modèle de programmation GPU	Très avancé	Très mature	Gratuit	Complexe	Sans mesure	+
GeNN [2]	Génération de code CUDA	Avancé	Mature	Gratuit	Assez facile	×200 vs 1 cœur CPU	+
Brian2CUDA [3]	Backend GPU de Brian2	Modéré	Plutôt récent	Gratuit	Très facile	×1000 vs Brian CPU	+
Solveur Julia [4]	Solveur d' EDO multi- GPU	Avancé	Récent	Gratuit	Facile	×20–100 vs JAX/PyTorch	+
SimHH [5]	Multi- GPU (MPI)	Très avancé	Tout récent	Exige un cluster	Complexe	×150 vs CPU mono-fil	-

Couleur	Signification
	++ Meilleur cas
	+ Bon cas
	- Cas plutôt moyen
	– Cas moyen
	Non concerné

Bibliographie

- [1] J. Nickolls, I. Buck, M. Garland, K. Skadron, « Scalable Parallel Programming with CUDA », *ACM Queue*, vol. 6, n° 2, p. 24–32, mars 2008. DOI : 10.1145/1365490.1365500.
- [2] E. Yavuz, J. Turner, T. Nowotny, « GeNN : a code generation framework for accelerated brain simulations », *Scientific Reports*, vol. 6, article 18854, 2016. DOI : 10.1038/srep18854.
- [3] D. Alevi, M. Stimberg, H. Sprekeler, K. Obermayer, M. Augustin, « Brian2CUDA : Flexible and Efficient Simulation of Spiking Neural Network Models on GPUs », *Frontiers in Neuroinformatics*, vol. 16, article 883700, 2022. DOI : 10.3389/fninf.2022.883700.
- [4] U. Utkarsh, V. Churavy, Y. Ma, T. Besard, P. Srisuma, T. Gymnich, A. R. Gerlach, A. Edelman, G. Barbastathis, R. D. Braatz, C. Rackauckas, « Automated Translation and Accelerated Solving of Differential Equations on Multiple GPU Platforms », *Computer Methods in Applied Mechanics and Engineering*, vol. 419, 2024. DOI : 10.1016/j.cma.2023.116591. Prépublication : arXiv:2304.06835.
- [5] M. C. W. Engelen, J.-H. L. F. Betting, C. Strydis, « SimHH : A Versatile, Multi-GPU Simulator for Extended Hodgkin-Huxley Networks », *IEEE Access*, vol. 13, 2025. DOI : 10.1109/ACCESS.2025.3550444.